

C++ Coding Convention (Updated)

File Structure

- Header files (.h / .hpp) contain: * Class declarations * Function prototypes * Constants, enums, and type definitions - Source files (.cpp) contain: * Function and method implementations * Internal (private) helpers Each header must use include guards or #pragma once:

Naming Conventions

- Classes / Structs / Enums → PascalCase - Methods / Functions → camelCase - Variables * Local variables → camelCase * Member variables → camelCase_ (trailing underscore) * Global variables → gCamelCase - Constants / Macros → ALL_CAPS_WITH_UNDERSCORES - Namespaces → lowercase

Method Documentation (Mandatory)

■■ Rule: Every method MUST begin with the following description block. This is mandatory for all functions (public, private, static, free functions). /*****

Method: (method name) Description: (Describe the main function of method)

*****/ Example:

/******/ Method: calculateTotal Description: Calculates the total price of items in the shopping cart, including tax.

*****/

Formatting & Style

- Indentation → 4 spaces (no tabs) - Braces → K&R; style - Line length → max 100–120 chars - One declaration per line

Best Practices

- Avoid raw pointers → Use smart pointers (unique_ptr, shared_ptr) or references - Use const correctness - Prefer enum class over raw enums - Initialize variables at declaration - Use override for virtual methods - Avoid 'using namespace std;' in headers - Error handling → Use exceptions or error codes consistently - Logging and debugging → Follow project-wide logging standards

Include Guard Example

```
#ifndef PROJECT_MODULE_CLASSNAME_H
#define PROJECT_MODULE_CLASSNAME_H
...
#endif
```

Class Example

```
class ShoppingCart {
public:
    /*****
    Method: addItem
```

```

        Description: Adds an item with its price to the cart.
        *****/
void addItem(const std::string& name, double price);

/*****
    Method: calculateTotal
    Description: Calculates the total cost of all items
                 in the cart, including tax.
    *****/
double calculateTotal(double taxRate) const;

private:
    std::vector<std::pair<std::string, double>> items_;
};

```